

학번	30720
이름	안재민
탐구주제	칸토어 계단 함수의 n 차 근사를 활용한 Neural ODE 기반 윤리적 강건성 필터 설계

아래의 평가요소 및 채점기준을 참고하여 작성하세요.

1. 서론

가. 탐구동기

2024년 이후 ChatGPT, Claude, Gemini 등 대규모 언어모델이 일상에 깊이 들어오며, 이 모델들이 유해한 답변을 생성하는 사례가 꾸준히 보도되고 있다. 특히 탈옥이라 불리는 일련의 기법들을 통하여 모델들의 안전장치를 우회하는 문제는 AI 기업들의 최우선 해결 과제로 부상했다.

평소 수학에 흥미를 가지고 다양한 이론을 접하던 중 칸토어 집합과 악마의 계단이라고도 불리는 병리적 함수의 일종인 칸토어 함수를 접하게 되었다. 칸토어 집합과 칸토어 함수의 수학적 의미를 찾아보고 공부하며 이 함수의 독특한 성질에 강한 호기심을 느꼈다. 거의 모든 곳에서 도함수가 0이면서도 0에서 1까지 연속적으로 증가하는 이 함수가, 거의 모든 상황에서 안정적이면서 극히 미세한 경계에서도 판단이 전환되는 이상적인 인공지능 안전 필터와 구조적으로 닮아있다는 생각이 들었다.

이에 미적분 교과에서 배운 도함수, 정적분 등의 개념을 활용하여 칸토어 함수가 인공지능 정렬 문제에 어떻게 수학적으로 기여할 수 있는지에 대하여 탐구해보고자 한다.

나. 탐구 필요성

현재 인공지능 정렬 기법의 대부분은 학습 데이터를 통한 통계적 접근인 RLHF, DPO 등의 기법에 의존한다. 이러한 방법은 학습데이터에 존재하지 않는 새로운 유형의 위험에 취약하다는 구조적 한계를 지닌다. 수학적 구조에 기반한 정렬 메커니즘은 데이터에 의존하지 않으므로 이러한 한계를 보완할 수 있다.

2. 본문

가. 배경이론(방정식/함수)

1) 배경이론

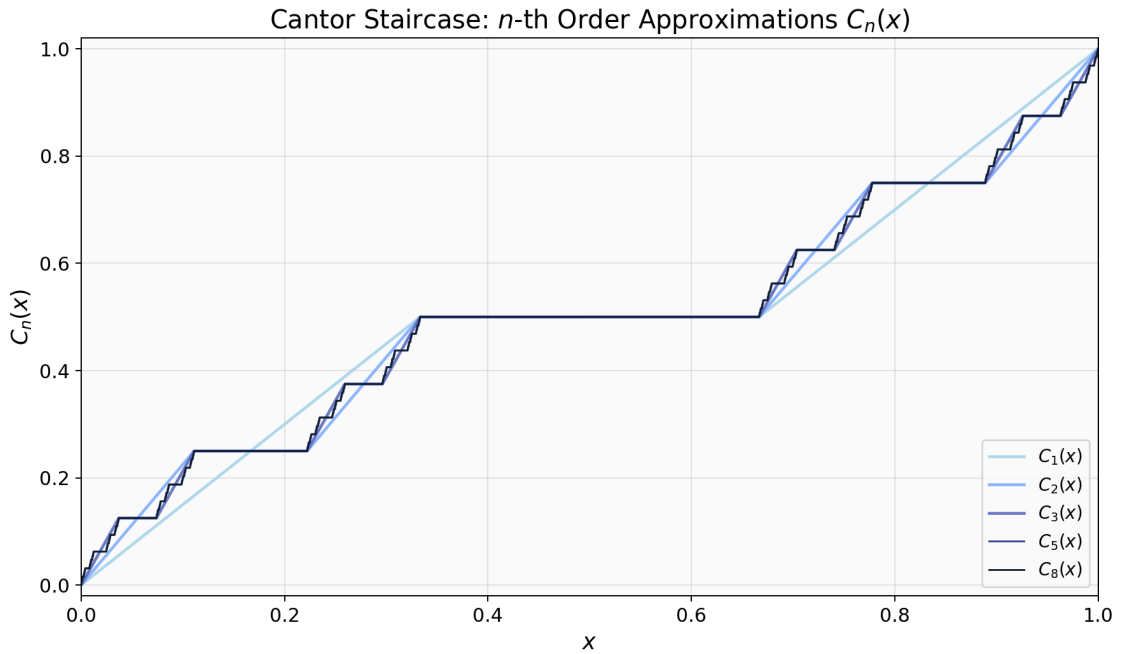
가) 칸토어 계단 함수

칸토어 계단함수 $C(x)$ 는 구간 $[0, 1]$ 에서 다음과 같이 재귀적으로 구성된다. 먼저 0차 근사를 항등함수 $C_0(x) = x$ 로 정의한 뒤, 각 단계에서 비평탄 구간의 가운데 $\frac{1}{3}$ 을 제거하고 해당 구간에서 함수값을 상수로 고정한다.

n 차 근사 $C_n(x) = x$ 의 재귀적 정의는 다음과 같이 표현 가능하다:

$$C_n(x) = \begin{cases} \frac{1}{2}C_{n-1}(3x) & 0 \leq x < \frac{1}{3} \\ \frac{1}{2} & \frac{1}{3} \leq x \leq \frac{2}{3} \\ \frac{1}{2} + \frac{1}{2}C_{n-1}(3x-2) & \frac{2}{3} < x \leq 1 \end{cases}$$

n 이 증가할수록 평탄 구간의 총 측도는 $1 - (\frac{2}{3})^n$ 으로 증가하여, $n \rightarrow \infty$ 에 수렴한다. 즉 극한 함수 $C(x)$ 는 거의 모든 곳에서 도함수가 0이면서 $C(0) = 0$ 에서 $C(1) = 1$ 까지 연속적으로 증가하는 특이 연속함수이다.



[그림 1] 참조: C_1, C_2, C_3, C_5, C_8 근사의 그래프를 중첩하면, n 이 커질수록 계단 구조가 점점 촘촘해지는 것을 확인할 수 있다.

나) KL-발산

두 확률분포 P 와 Q 사이의 KL-발산은 다음과 같이 정의 할 수 있다:

$$D_{KL}(P|Q) = \int_{-\infty}^{\infty} p(x) \ln \frac{p(x)}{q(x)}, dx$$

$P = N(\mu, \sigma^2)$ 이고 $Q = N(0, 1)$ 일 때, 이를 직접 계산하면:

$$D_{KL} = -\ln \sigma + \frac{\sigma^2 + \mu^2}{2} - \frac{1}{2}$$

특히 $\sigma = 1$ 로 고정하면:

$$D_{KL}(\mu) = \frac{\mu^2}{2}$$

이는 모델의 출력 분포(P)가 안전 기준 분포(Q)에서 벗어날수록 **KL-발산** 값이 2차 함수적으로 증가함을 의미한다. 이번 탐구에서는 이 **KL-발산**을 퍼텐셜 에너지로 해석하여, 모델의 상태를 안전한 방향으로 끌어당기는 힘으로서 사용한다.

[그림 3 좌측] 참조.

다) Neural ODE

Neural ODE는 신경망의 은닉 상태 $h(t)$ 의 변화를 이산적 층이 아닌 연속적 미분방정식으로 모델링하는 체계이다:

$$\frac{dh(t)}{dt} = f_{\theta}(h(t))$$

이번 탐구에서는 이 프레임워크에 **KL-발산 퍼텐셜**과 **칸토어 필터**를 결합하여 다음과 같은 시스템을 설계했다:

$$\frac{dh}{dt} = -\alpha(h - h_{\text{safe}}) + C'_n(h) \cdot p(t)$$

여기서:

- $-\alpha(h - h_{\text{safe}})$: **KL-발산** 기울기로부터 유도된 복원력
- $C'_n(h)$: 칸토어 필터
- $p(t) = A \sin(\omega t)$: 외부 섭동

2) 배경이론 함수(또는 방정식)의 도함수

(1) 칸토어 계단 함수의 도함수 유도

1차 근사 $C_1(x)$ 를 구간별로 정의하면:

$$C_1(x) = \begin{cases} \frac{3}{2}x & 0 \leq x < \frac{1}{3} \\ \frac{1}{2} & \frac{1}{3} \leq x \leq \frac{2}{3} \\ \frac{3}{2}x - \frac{1}{2} & \frac{2}{3} < x \leq 1 \end{cases}$$

각 구간에서 도함수를 구하면:

$$C'_1(x) = \begin{cases} \frac{d}{dx} \left(\frac{3}{2}x \right) = \frac{3}{2} & 0 < x < \frac{1}{3} \\ \frac{d}{dx} \left(\frac{1}{2} \right) = 0 & \frac{1}{3} < x < \frac{2}{3} \\ \frac{d}{dx} \left(\frac{3}{2}x - \frac{1}{2} \right) = \frac{3}{2} & \frac{2}{3} < x < 1 \end{cases}$$

2차 근사 $C_2(x)$ 에서는, C_1 의 비평탄 구간 $[0, 1/3]$ 과 $[2/3, 1]$ 각각에 동일한 삼등분-고정 과정이 반복된다. 예를 들어 $[0, \frac{1}{3}]$ 구간의 세부 구조는:

$$C_2(x) = \begin{cases} \frac{9}{4}x & 0 \leq x < \frac{1}{9} \\ \frac{1}{4} & \frac{1}{9} \leq x \leq \frac{2}{9} \\ \frac{9}{4}x - \frac{1}{4} & \frac{2}{9} < x \leq \frac{1}{3} \end{cases}$$

비평탄 구간에서의 도함수:

$$C'_2(x) = \frac{9}{4} = \left(\frac{3}{2} \right)^2 \quad (\text{Non-flat})$$

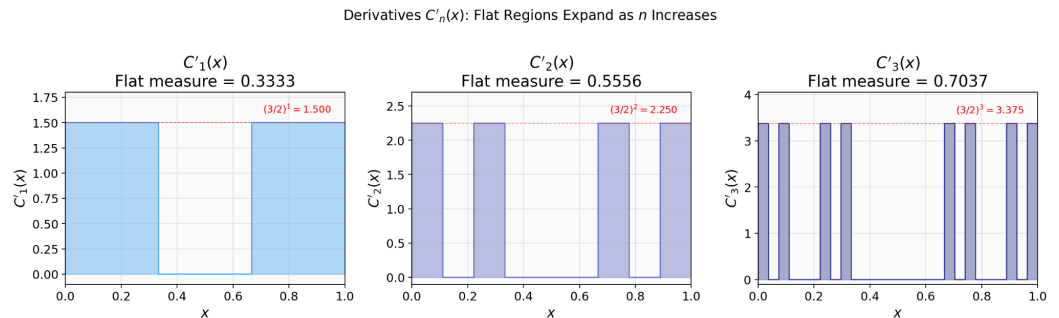
이 과정을 귀납적으로 일반화하면, n 차 근사의 도함수는 다음과 같은 이값 함수가 된다:

$$C'_n(x) = \begin{cases} \left(\frac{3}{2} \right)^n & x \in \text{non-flat intervals (measure } (2/3)^n) \\ 0 & x \in \text{flat intervals (measure } 1 - (2/3)^n) \end{cases}$$

여기서 n 이 증가할수록 비평탄 구간의 측도는 $(2/3)^n \rightarrow 0$ 이고, 해당 구간에서의 도함수 값은 $(3/2)^n \rightarrow \infty$ 이다. 두 값의 곱은 항상:

$$(2/3)^n \times (3/2)^n = 1$$

이 성질은 정적분을 통해 검증할 수 있다 (후술).



[그림 2] 참조: C'_1, C'_2, C'_3 의 그래프에서 평탄 구간이 점차 확장되고, 비평탄 구간의 높이가 증가하는 패턴이 명확히 드러난다.

(2) KL-발산의 도함수 유도

$\sigma = 1$ 로 고정한 KL-발산 $D_{KL}(\mu) = \frac{\mu^2}{2}$ 에 대해:

$$\frac{d}{d\mu} D_{KL} = \frac{d}{d\mu} \left(\frac{\mu^2}{2} \right) = \mu$$

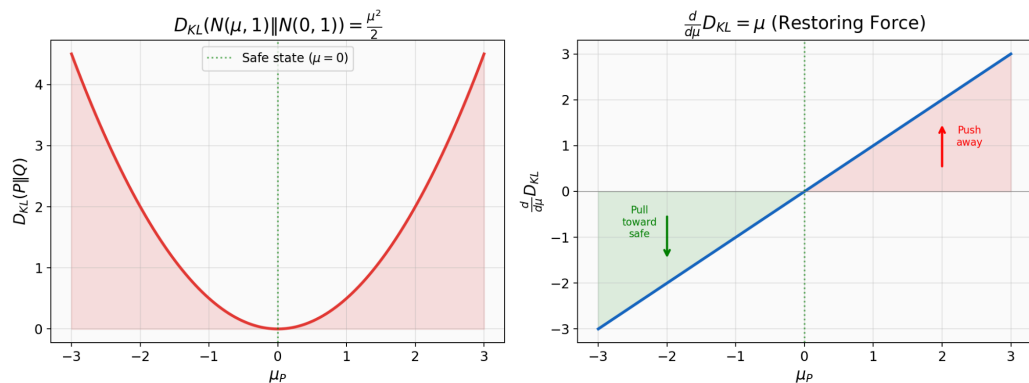
여기서 $\mu > 0$ (모델이 안전 분포보다 오른쪽으로 치우쳤을 때)이면 $\frac{dD_{KL}}{d\mu} > 0$

이므로, KL-발산을 줄이려면 μ 를 감소시켜야 한다. 즉, $-\frac{dD_{KL}}{d\mu} = -\mu$ 는 원점(안전 상태)을 향한 복원력으로 해석할 수 있다.

일반적인 경우 $\sigma \neq 1$ 에서의 편도함수:

$$\frac{\partial D_{KL}}{\partial \mu} = \mu, \quad \frac{\partial D_{KL}}{\partial \sigma} = -\frac{1}{\sigma} + \sigma$$

$\frac{\partial D_{KL}}{\partial \sigma} = 0$ 을 풀면 $\sigma = 1$ 이 극솟값임을 확인할 수 있다. 이는 모델의 출력 분포가 안전 기준 분포와 동일한 분산을 가질 때 KL-발산이 최소화됨을 의미한다.



[그림 3 우측] 참조: $\frac{dD_{KL}}{d\mu} = \mu$ 의 그래프에서, 양의 영역과 음의 영역이 각각 척력과 인력으로 작용하는 것을 시각적으로 확인할 수 있다.

(3) 정적분을 통한 칸토어 도함수 검증 미적분의 기본정리에 의해:

$$\int_0^x C'_n(t), dt = C_n(x) - C_n(0) = C_n(x)$$

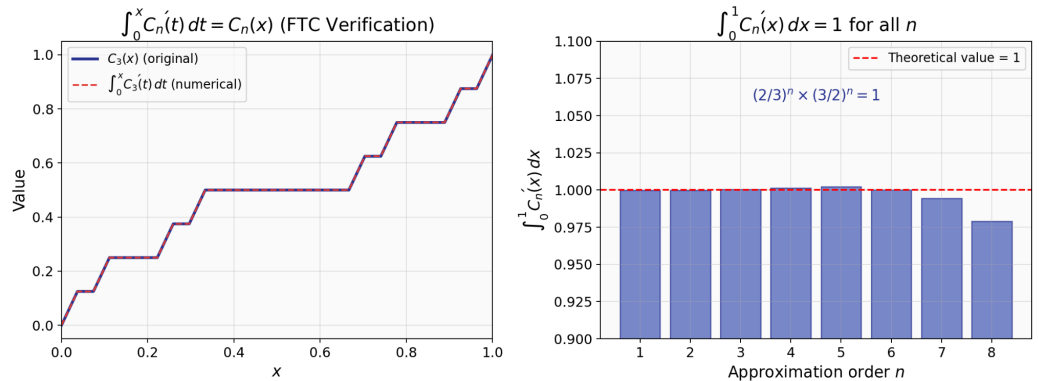
특히 $x = 1$ 일 때:

$$\int_0^1 C'_n(t), dt = C_n(1) - C_n(0) = 1 - 0 = 1$$

이 등식은 다음과 같이 직접 검증 가능하다. $C'_n(x)$ 는 총 속도 $\left(\frac{2}{3}\right)^n$ 인 비평탄 구간에서 값 $\left(\frac{3}{2}\right)^n$ 을 갖고, 나머지에서 0이므로:

$$\int_0^1 C'_n(t), dt = \left(\frac{3}{2}\right)^n \times \left(\frac{2}{3}\right)^n = 1$$

여기서 비평탄 구간의 속도가 아무리 작아져도, 해당 구간에서의 도함수 값이 정확히 그만큼 커지므로 정적분 값이 보존된다. 이는 칸토어 함수의 특징인 질량을 잃지 않으면서 점점 더 좁은 영역에 집중시키는 프랙탈적 재분배의 수학적 표현으로 해석할 수 있다.



[그림 3 좌측] 참조: $\int_0^x C'_3(t), dt$ 의 수치 적분 결과가 $C_3(x)$ 와 정확히 일치한다.

[그림 3 우측] 참조: $n = 1, 2, \dots, 8$ 모두에 대해 $\int_0^1 C'_n(x), dx = 1$ 이 성립한다.

나. 탐구방법

이번 탐구의 핵심인 칸토어 필터가 **Neural ODE** 궤적의 섭동 강건성을 향상시킨다라는 것을 검증하기 위해 다음의 비교 시뮬레이션을 설계하였다.

시뮬레이션 설정:

- 상태공간 : $h \in [0, 1]$, 안전 상태 $h_{\text{safe}} = 0.5$ (칸토어 함수 C_1 의 주 평탄 구간 $[1/3, 2/3]$ 의 중심)
- 초기 조건 : $h(0) = 0.15$ (안전 영역 바깥에서 출발)
- 외부 섭동 : $p(t) = A \sin(\omega t)$ ($A = 1.5, \omega = 4.0$)
- 복원력 상수 : $\alpha = 0.8$

비교 모델:

모델	미분방정식	설명
(A) 필터 없음	$\frac{dh}{dt} = -\alpha(h - 0.5) + A \sin(\omega t)$	섭동이 여과 없이 전달됨.
(B) C_3 필터	$\frac{dh}{dt} = -\alpha(h - 0.5) + \frac{C'_3(h)}{(3/2)^3} A \sin(\omega t)$	평탄구간 측도 70.4% 기록.
(C) C_5 필터	$\frac{dh}{dt} = -\alpha(h - 0.5) + \frac{C'_5(h)}{(3/2)^5} A \sin(\omega t)$	평탄구간 측도 86.8% 기록.

칸토어 필터의 동작 원리인 $C'_n(h)$ 를 $(3/2)^n$ 으로 정규화하여 게이트 값을 $[0, 1]$ 범위로 조정한다. h 가 평탄 구간에 있으면 게이트 = 0으로 섭동이 온나전 차단되고, 비평탄 구간에서는 게이트 = 1로 섭동이 통과된다.

측정 지표:

- 안전 영역 체류 비율 : 전체 시뮬레이션 시간 중 $h(t) \in [1/3, 2/3]$ 인 시간의 비율
- 위상 초상 : $(h, dh/dt)$ 평면에서의 궤적 분포

구현 환경: Julia, [DifferentialEquations.jl](#), [Plots.jl](#)

다. 탐구내용

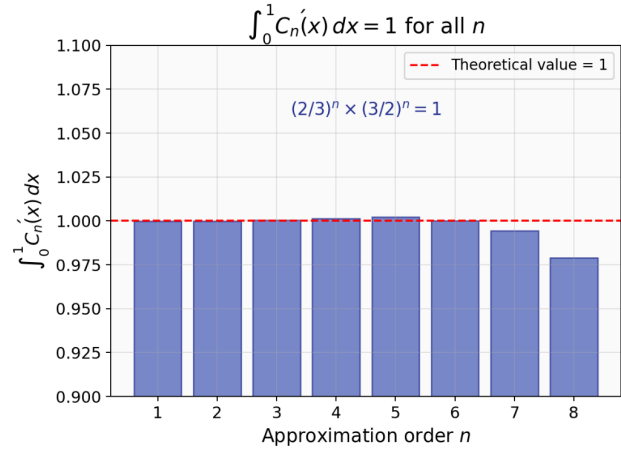
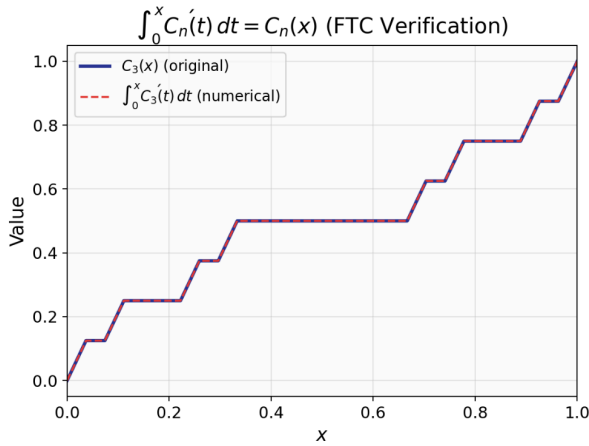
(1) 칸토어 계단 함수의 시각화 및 분석

[그림 1]은 $C_1(x)$ 부터 $C_8(x)$ 까지의 근사를 중첩한 그래프이다. n 이 증가할수록 계단 구조가 세밀해지며, C_8 은 이미 극한 함수 $C(x)$ 와 시각적으로 거의 구분되지 않는다.

[그림 2]는 C'_1, C'_2, C'_3 의 그래프이다. 주된 내용은 다음과 같다.

- $C'_1(x)$: 평탄 구간 측도 $1/3 \approx 0.333$, 비평탄 높이 $3/2 = 1.5$
- $C'_2(x)$: 평탄 구간 측도 $5/9 \approx 0.556$, 비평탄 높이 $9/4 = 2.25$
- $C'_3(x)$: 평탄 구간 측도 $19/27 \approx 0.704$, 비평탄 높이 $27/8 = 3.375$

n 이 커질수록 도함수가 0인 영역이 빠르게 전체 구간을 지배하며, 이는 칸토어 필터가 점점 더 넓은 범위의 입력에 대하여 섭동을 차단함을 뜻한다.



[그림 5]는 이 경향을 정량적으로 보여준다. 평탄 구간 측도 $1 - (2/3)^n$ 은 $n = 10$ 에서 이미 98.3%에 도달한다.

(2) 정적분을 통한 미적분의 기본정리 검증

[그림 4 좌측]에서 $C_3'(x)$ 의 수치 부정적분 $\int_0^x C_3'(t) dt$ 를 구한 결과가 원래 함수 $C_3(x)$ 와 정확히 일치함을 확인할 수 있다. 여기서 미적분의 기본정리가 칸토어 계단 함수의 근사에 대해서도 성립함을 알 수 있다.

[그림 4 우측]에서 $n = 1$ 부터 $n = 8$ 까지 모든 차수에 대해 $\int_0^1 C_n'(x) dx = 1$ 이 수치적으로 성립함을 확인할 수 있다. 이 결과는 $(2/3)^n \times (3/2)^n = 1$ 이라는 대수적 항등식의 해석기하학적 의미를 보여주는데, 비평탄 구간의 폭이 줄어드는 만큼 높이가 정확히 증가하여, 넓이가 보존된다.

(3) KL-발산의 퍼텐셜 해석

[그림 3 좌측]에서 $D_{KL}(\mu) = \mu^2/2$ 는 $\mu = 0$ (안전 상태)에서 최솟값 0을 가지는 아래로 볼록한 포물선이다. 이를 퍼텐셜 에너지로 해석하면, $\mu = 0$ 은 에너지가 최소인 안정 평형점이다.

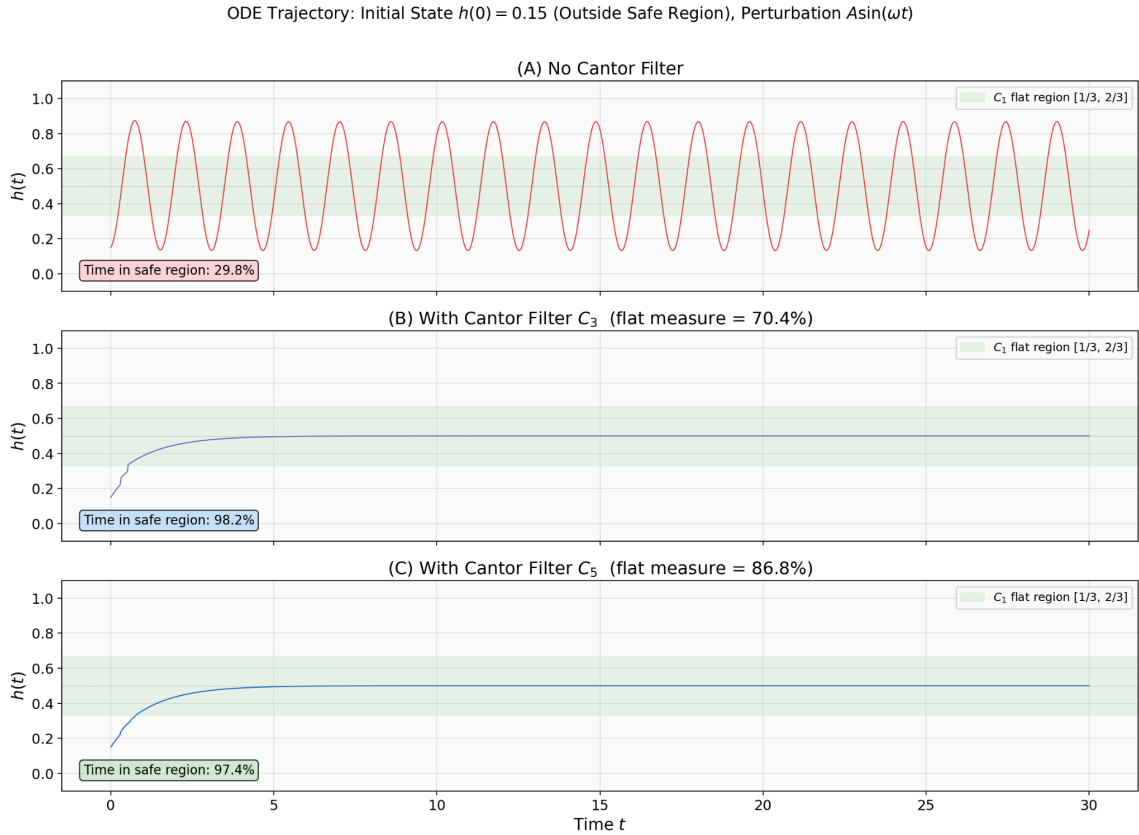
[그림 3 우측]에서 도함수 $dD_{KL}/d\mu = \mu$ 는 원점을 기준으로 부호가 바뀌는 복원력을 나타낸다:

- $\mu > 0$: 양의 기울기 $\rightarrow -dD_{KL}/d\mu < 0 \rightarrow$ 왼쪽(안전 방향)의 인력
- $\mu < 0$: 음의 기울기 $\rightarrow -dD_{KL}/d\mu > 0 \rightarrow$ 오른쪽(안전 방향)의 인력

이 복원력이 ODE 시스템의 $-\alpha(h - h_{\text{safe}})$ 항에 대응된다.

(4) ODE 궤적 비교 시뮬레이션 결과

[그림 6]은 세 모델의 궤적을 비교한 시각화이다.



모델 (A), 필터 없음 : 외부 섭동 $A\sin(\omega t)$ 가 여과 없이 전달되어 $h(t)$ 가 큰 진폭으로 진동한다. 안전 영역 $[1/3, 2/3]$ 체류 비율은 약 29.8%에 불과하다. 초기 위치 $h(0) = 0.15$ 에서 안전 영역으로 복원되려 하지만, 섭동의 힘이 복원력과 비슷하므로 지속적으로 안전 영역을 이탈한다.

모델 (B), C_3 필터 : 궤적이 안전 영역 바깥($h = 0.15$)에서 시작하여, 비평탄 구간을 빠르게 통과한 뒤 안전 영역의 평탄 구간에 진입한다. 일단 평탄 구간에 도달하면 $C'_3(h) = 0$ 이므로 섭동 항이 완전히 차단되어, 순수한 복원력 $-\alpha(h - 0.5)$ 만 작용한다. 안전 영역 체류 비율은 약 98.2%.

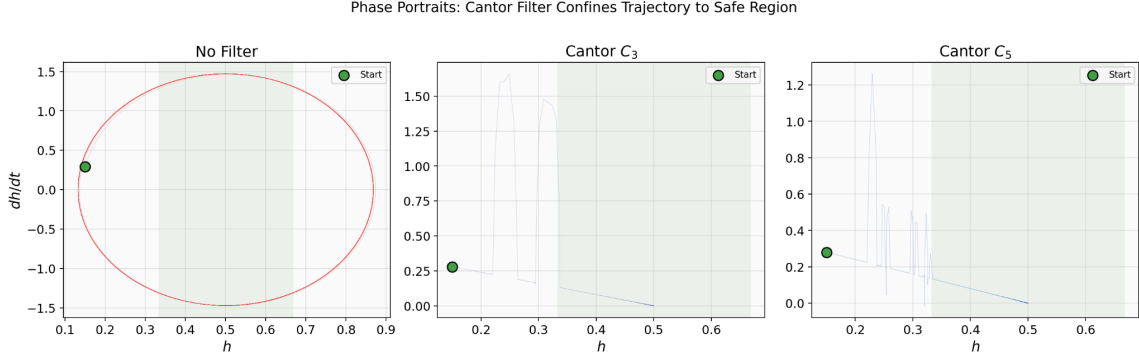
모델 (C), C_5 필터 : C_3 보다 평탄 구간이 더 넓으므로(86.8% vs 70.4%), 비평탄 구간을 통과하는 시간이 더 짧다. 안전 영역 체류 비율: 약 97.4%.

칸토어 필터가 적용된 두 모델 모두, 초기에 안전영역 바깥에서 출발했음에도 빠르게 안전 영역에 수렴하여 이후 섭동에 대한 완전한 강건성을 보인다. 이를 통해 칸토어

함수의 도함수가 평탄 구간에서 정확히 0이라는 수학적 성질을 가지는 것이 공학적 강건으로 이어진다는 것을 알 수 있었다.

(5) 위상 초상 분석

[그림 7]은 $(h, dh/dt)$ 평면에서의 궤적을 보여준다.



필터 없음 : 궤적이 넓은 타원을 그리며 상태 공간 전체를 순환한다. h 의 범위가 약 $[0.1, 0.9]$ 로, 안전 영역을 크게 벗어난다.

C_3 필터 : 초기 과도 구간 이후 궤적이 안전 영역 내부의 좁은 점으로 수렴한다. $(h, dh/dt) \approx (0.5, 0)$ 에 밀집되어, 시스템이 안정 평형에 도달했음을 보여준다.

C_5 필터 : C_3 필터와 유사하나, 과도 구간이 더 짧고 수렴이 더 빠르다. 위상 공간에서의 궤적 분포가 가장 좁다.

위상 초상은 칸토어 필터가 시스템의 끌개 구조를 근본적으로 변화시킴을 보여준다. 필터없는 시스템의 한계순환이, 필터가 적용되면 안정 고정점으로 전환된다.

3. 결론

가. 느낀점

이번 탐구를 통하여 미적분이 단순한 계산 도구를 넘어, 현실 문제 구조를 설계하는 언어가 될 수 있음을 체감하였다. 특히 칸토어 계단 함수의 도함수가 거의 모든 곳에서 0이라는 성질을 처음 알게 되었을 때는 신기한 수학적 특징이라고만 느꼈으나, 이를 인공지능 안전 필터의 맥락에 놓으니 거의 모든 입력에 대하여 섭동을 차단한다는 강력한 공학적 의미로 바뀌었다.

미적분의 기본정리를 통하여 $\int_0^1 C'_n(x) dx = 1$ 이 모든 n 에 대하여 성립함을 확인하는 과정에서는, $(2/3)^n \times (3/2)^n = 1$ 이라는 단순한 대수적 항등식이 평탄 구간의 확장과 도함수의 높이 증가가 정확히 상쇄된다는 기하학적 의미를 품고있다는 사실에 경이로움을 느꼈다.

또한 시뮬레이션을 설계하고 또한 코드로 구현하는 과정에서, 수학적 이론이 컴퓨터 시뮬레이션으로 검증될 때 느껴지는 확신의 무게를 실감하였다. 도함수의 부호와 값이 궤적의 거동을 결정한다는 미적분의 기본원리가, 화면 위의 그래프에서 눈으로 확인되는 경험은 수학에 대한 신뢰를 깊게 해 주었다.

나. 제언

이번 탐구는 1차원 상태 공간에서의 개념 검증에 해당한다. 다음과 같이 향후 발전 방향을 제시한다.

첫째, 실제 LLM의 은닉 상태는 수천 차원 이상의 고차원 벡터이므로, 칸토어 필터를 고차원으로 확장하는 수학적 체계가 필요하다. 칸토어 집합의 고차원 일반화인 칸토어 먼지의 성질을 활용하는 연구를 구상중이다.

둘째, 칸토어 함수의 근사 차수를 n 으로 고정하는 대신, 입력의 위험도에 따라 n 을 동적으로 조절하는 적응형 필터를 고려중이다.

셋째, 본 탐구에서 사용한 KL-발산 퍼텐셜을 변분법의 프레임워크에서 최적화하여 오일러-라그랑주 방정식을 통해 최적 안전 경로를 해석적으로 유도하는 탐구를 구상중이다.

[참고문헌]

- Chen, R. T., et al. (2018). "Neural Ordinary Differential Equations." *Advances in Neural Information Processing Systems*, 31.
- Bai, Y., et al. (2022). "Constitutional AI: Harmlessness from AI Feedback." *Anthropic Technical Report*.
- Dovgoshey, O., et al. (2006). "The Cantor Function." *Expositiones Mathematicae*, 24(1), 1-37.
- Kullback, S., & Leibler, R. A. (1951). "On Information and Sufficiency." *The Annals of Mathematical Statistics*, 22(1), 79-86.

[부록, Julia 코드]

Julia 코드	<pre>using DifferentialEquations using Plots using LaTeXStrings gr() const OUTDIR = joinpath(@__DIR__, "figures") mkpath(OUTDIR)</pre>
----------	--

```

# -----
# §1 핵심 함수 정의
# -----

"""
cantor_staircase(x, n) → Float64


$$C_n(x) = \begin{cases} \frac{1}{2} C_{n-1}(3x) & (0 \leq x < 1/3) \\ \frac{1}{2} & (1/3 \leq x \leq 2/3) \\ \frac{1}{2} + \frac{1}{2} C_{n-1}(3x - 2) & (2/3 < x \leq 1) \end{cases}$$

"""
function cantor_staircase(x::Real, n::Int)::Float64
    n == 0 && return Float64(x)
    x ≤ 0 && return 0.0
    x ≥ 1 && return 1.0

    if x < 1/3
        return 0.5 * cantor_staircase(3x, n - 1)
    elseif x ≤ 2/3
        return 0.5
    else
        return 0.5 + 0.5 * cantor_staircase(3x - 2, n - 1)
    end
end

"""
cantor_derivative(x, n) → Float64


$$C'_n(x) = \begin{cases} (3/2)^n & (\text{비평탄 구간, 총 측도 } (2/3)^n) \\ 0 & (\text{평탄 구간, 총 측도 } 1 - (2/3)^n) \end{cases}$$

"""

```

```

function cantor_derivative(x::Real, n::Int)::Float64
    (x ≤ 0 || x ≥ 1) && return 0.0
    xf = Float64(x)
    for _ in 1:n
        if 1/3 < xf < 2/3
            return 0.0
        elseif xf ≤ 1/3
            xf = 3xf
        else
            xf = 3xf - 2
        end
    end
    return (3/2)^n
end

"""
    kl_divergence(μ; σ=1.0) → Float64

    KL(N(μ, σ²) || N(0, 1))
"""
kl_divergence(μ; σ=1.0) = -log(σ) + (σ^2 + μ^2) / 2 - 0.5

"""
    kl_derivative_mu(μ) → Float64

    ∂/∂μ KL(N(μ, 1) || N(0, 1)) = μ
"""
kl_derivative_mu(μ) = μ

function plot_fig1()
    xs = range(0, 1; length=8000)

```

```

orders = [1, 2, 3, 5, 8]
colors = [:lightskyblue, :cornflowerblue, :slateblue, :midnightblue,
:black]

p = plot(
    xlabel = L"x",
    ylabel = L"C_n(x)",
    title = L"\textrm{Cantor\ Staircase:\ }n\textrm{-th\ Order\ }
Approximations\ }C_n(x)",
    xlims = (0, 1),
    ylims = (-0.02, 1.02),
    legend = :bottomright,
    grid = true,
    gridalpha = 0.3,
    size = (900, 540),
    dpi = 180,
)

for (i, n) in enumerate(orders)
    ys = cantor_staircase.(xs, n)
    plot!(p, xs, ys;
        label = latexstring("C_{\$n}(x)"),
        color = colors[i],
        linewidth = n < 5 ? 1.8 : 1.2,
        alpha = 0.9,
    )
end

savefig(p, joinpath(OUTDIR, "fig1_cantor_staircase.png"))
println("✓ Figure 1 saved")

return p
end

```

```

function plot_fig2()
    xs = range(0.001, 0.999; length=15000)
    ns = [1, 2, 3]
    colors = [:steelblue1, :slateblue, :midnightblue]

    plots = []
    for (i, n) in enumerate(ns)
        dy = cantor_derivative.(xs, n)
        flat_meas = 1 - (2/3)^n
        peak = (3/2)^n

        p = plot(xs, dy;
            seriestype = :steppre,
            fill       = (0, 0.35, colors[i]),
            color       = colors[i],
            linewidth   = 0.8,
            xlabel      = L"x",
            ylabel      = latexstring("C'_{\$n}(x)"),
            title       = latexstring("C'_{\$n}(x)\\quad\\text{\\rm{Flat\\measure}}=\$(round(flat_meas; digits=4))"),
            ylims       = (-0.1, peak * 1.25),
            xlims       = (0, 1),
            legend      = false,
            grid        = true,
            gridalpha   = 0.3,
        )
        hline!(p, [peak]; color=:red, linestyle=:dash, alpha=0.5,
            linewidth=0.8)
        annotate!(p, 0.95, peak * 1.08,
            text(latexstring("(3/2)^{\$n}=\$(round(peak; digits=3))"), 9,
            :red, :right))
    end
end

```

```

        push!(plots, p)
    end

    combined = plot(plots...;
        layout      = (1, 3),
        size        = (1350, 400),
        dpi         = 180,
        plot_title  = L"\textrm{Derivatives\ }C'_n(x)\textrm{:}\ Flat\
Regions\ Expand\ as\ }n\textrm{\ Increases}",
    )
    savefig(combined, joinpath(OUTDIR, "fig2_cantor_derivatives.png"))
    println("✓ Figure 2 saved")
    return combined
end

function plot_fig3()
    dx = 0.0001
    xs_int = 0:dx:1

    n_demo = 3
    dy_int = cantor_derivative.(xs_int, n_demo)
    integral_cumsum = cumsum(dy_int) .* dx

    xs_orig = range(0, 1; length=5000)
    ys_orig = cantor_staircase.(xs_orig, n_demo)

    pl = plot(xs_orig, ys_orig;
        color      = :midnightblue,
        linewidth  = 2.5,
        label      = latexstring("C_{\$n_demo}(x)\textrm{\ (original)}"),
        xlabel     = L"x",

```



```

        ylabel      = "Value",
        title       = L"\int_0^x C'_3(t)\,dt = C_3(x)\textrm{\ (FTC\
Verification)}",
        grid        = true,
        gridalpha   = 0.3,
    )
    plot!(p1, collect(xs_int), integral_cumsum;
        color        = :red,
        linewidth    = 1.5,
        linestyle    = :dash,
        label        = L"\int_0^x C'_3(t)\,dt\textrm{\ (numerical)}",
    )

    ns_range = 1:8
    integrals = Float64[]
    for n in ns_range
        dy = cantor_derivative.(xs_int, n)
        push!(integrals, sum(dy) * dx)
    end

    p2 = bar(collect(ns_range), integrals;
        color        = :slateblue,
        alpha        = 0.8,
        linecolor    = :midnightblue,
        xlabel       = L"\textrm{Approximation\ order\ }n",
        ylabel       = L"\int_0^1 C'_n(x)\,dx",
        title        = L"\int_0^1 C'_n(x)\,dx = 1\ \textrm{for\ all\ }n",
        ylims        = (0.9, 1.1),
        legend       = :topright,
        label        = "",
        grid         = true,
        gridalpha    = 0.3,
    )

```

```

)
hline!(p2, [1.0]; color=:red, linestyle=:dash, linewidth=1.5,
label="Theoretical = 1")
annotate!(p2, 4.5, 1.065,
text(L"(2/3)^n \times (3/2)^n = 1", 12, :midnightblue))

combined = plot(p1, p2;
layout = (1, 2),
size = (1200, 460),
dpi = 180,
)

savefig(combined, joinpath(OUTDIR, "fig3_integral_verification.png"))
println("✓ Figure 3 saved")
return combined
end

function plot_fig4()
μs = range(-3, 3; length=500)
kl_vals = kl_divergence.(μs)
dkl_vals = kl_derivative_mu.(μs)

p1 = plot(μs, kl_vals;
color = :red3,
linewidth = 2.5,
fill = (0, 0.12, :red3),
xlabel = L"\mu_P",
ylabel = L"D_{KL}(P || Q)",
title = L"D_{KL}(N(\mu,1) || N(0,1)) = \frac{\mu^2}{2}",
legend = :top,
label = "",
grid = true,
gridalpha = 0.3,

```

```

)
vline!(p1, [0]; color=:green, linestyle=:dot, alpha=0.6,
       label=L"\textrm{Safe\ state\ }(\mu=0)")

p2 = plot(μs, dkl_vals;
          color      = :royalblue,
          linewidth  = 2.5,
          xlabel     = L"\mu_P",
          ylabel     = L"\frac{d}{d\mu}D_{KL}",
          title      = L"\frac{d}{d\mu}D_{KL} = \mu\textrm{\textbackslash (Restoring\
Force))",
          label      = "",
          grid       = true,
          gridalpha  = 0.3,
          )

plot!(p2, μs, dkl_vals;
      fill      = (0, 0.12, :red3),
      fillrange = max.(dkl_vals, 0),
      color     = :transparent,
      label     = "",
      )

plot!(p2, μs, min.(dkl_vals, 0);
      fill      = (0, 0.12, :green),
      color     = :transparent,
      label     = "",
      )

hline!(p2, [0]; color=:gray, linewidth=0.5, label="")
vline!(p2, [0]; color=:green, linestyle=:dot, alpha=0.6, label="")

annotate!(p2, -2.3, -1.2, text("← Pull toward safe", 9, :green))
annotate!(p2, 2.3, 1.2, text("Push away →", 9, :red))

```

```

combined = plot(p1, p2;
    layout = (1, 2),
    size   = (1200, 460),
    dpi    = 180,
)
savefig(combined, joinpath(OUTDIR, "fig4_kl_divergence.png"))
println("Figure 4 saved")
return combined
end

const H_SAFE = 0.5
const  $\alpha$    = 0.8
const A PERT = 1.5
const  $\omega$     = 4.0
const TSPAN  = (0.0, 30.0)
const H0     = [0.15]

function ode_no_filter!(dh, h, p, t)
    dh[1] = - $\alpha$  * (h[1] - H_SAFE) + A PERT * sin( $\omega$  * t)
end

function make_ode_cantor(n::Int)
    function ode_cantor!(dh, h, p, t)
        hc = clamp(h[1], 0.001, 0.999)
        cd = cantor_derivative(hc, n)
        gate = cd / (3/2)^n # 정규화  $\rightarrow$  [0, 1]
        dh[1] = - $\alpha$  * (h[1] - H_SAFE) + gate * A PERT * sin( $\omega$  * t)
    end
    return ode_cantor!
end
end

```

```

function plot_fig5()

    prob_no = ODEProblem(ode_no_filter!, H0, TSPAN)
    prob_c3 = ODEProblem(make_ode_cantor(3), H0, TSPAN)
    prob_c5 = ODEProblem(make_ode_cantor(5), H0, TSPAN)

    sol_no = solve(prob_no, Tsit5(); saveat=0.005, dtmax=0.005)
    sol_c3 = solve(prob_c3, Tsit5(); saveat=0.005, dtmax=0.005)
    sol_c5 = solve(prob_c5, Tsit5(); saveat=0.005, dtmax=0.005)

    safe_lo, safe_hi = 1/3, 2/3

    function make_panel(sol, title_str, color, bg_color)
        h = sol[1, :]
        pct = round(100 * mean(safe_lo .≤ h .≤ safe_hi); digits=1)

        p = plot(sol.t, h;
            color      = color,
            linewidth  = 0.9,
            ylabel     = L"h(t)",
            title      = title_str,
            ylims      = (-0.1, 1.1),
            legend     = :topright,
            label      = "",
            grid       = true,
            gridalpha  = 0.3,
        )
        hspan!(p, [safe_lo, safe_hi];
            color=:palegreen, alpha=0.3,
            label=L"C_1\textrm{\ flat\ region\ }[1/3,\,2/3]")
        hline!(p, [H_SAFE]; color=:gray, linewidth=0.5, linestyle=:dot,

```

```

label="")

    annotate!(p, sol.t[end] * 0.02, 0.05,
              text("Time in safe region: $(pct)%", 10, :left))

    return p
end

mean(x) = sum(x) / length(x)

p1 = make_panel(sol_no, "(A) No Cantor Filter",
                :red3, :mistyrose)
p2 = make_panel(sol_c3, L"(B)\ \textrm{With\ Cantor\ Filter\ }C_3\
\textrm{(flat\ measure=70.4\%)}", :slateblue, :lavender)
p3 = make_panel(sol_c5, L"(C)\ \textrm{With\ Cantor\ Filter\ }C_5\
\textrm{(flat\ measure=86.8\%)}", :royalblue4, :honeydew)
plot!(p3; xlabel=L"\textrm{Time\ }t")

combined = plot(p1, p2, p3;
                layout = (3, 1),
                size = (1200, 850),
                dpi = 180,
                plot_title = L"\textrm{ODE\ Trajectory:\ Initial\
}h(0)=0.15\textrm{\ (Outside\ Safe\ Region)}",
                )

savefig(combined, joinpath(OUTDIR, "fig5_ode_comparison.png"))
println("✓ Figure 5 saved")
return (sol_no, sol_c3, sol_c5)
end

function plot_fig6(sol_no, sol_c3, sol_c5)
    safe_lo, safe_hi = 1/3, 2/3

```

```

function phase_panel(sol, title_str, color)
    h    = sol[1, :]
    dhdt = diff(h) ./ diff(sol.t)
    hm    = h[1:end-1]

    p = plot(hm, dhdt;
        color      = color,
        linewidth  = 0.35,
        alpha      = 0.7,
        xlabel     = L"h",
        title      = title_str,
        legend     = :topright,
        label      = "",
        grid       = true,
        gridalpha  = 0.3,
    )
    vspan!(p, [safe_lo, safe_hi]; color=:palegreen, alpha=0.2,
label="")
    scatter!(p, [h[1]], [dhdt[1]];
        color      = :green,
        markersize  = 7,
        markerstrokecolor = :black,
        markerstrokewidth = 1.2,
        label       = "Start",
    )
    return p
end

p1 = phase_panel(sol_no, "No Filter", :red3)
p2 = phase_panel(sol_c3, L"\textrm{Cantor\ }C_3", :slateblue)
p3 = phase_panel(sol_c5, L"\textrm{Cantor\ }C_5", :royalblue4)

```

```

plot!(p1; ylabel=L"dh/dt")

combined = plot(p1, p2, p3;
    layout      = (1, 3),
    size        = (1400, 430),
    dpi         = 180,
    plot_title  = L"\textrm{Phase\ Portraits:\ Cantor\ Filter\
Confines\ Trajectory}",
    )
savefig(combined, joinpath(OUTDIR, "fig6_phase_portrait.png"))
println("✓ Figure 6 saved")
return combined
end

function plot_fig7()
    ns = 1:15
    flat_measures = [1 - (2/3)^n for n in ns]
    peak_derivs   = [(3/2)^n     for n in ns]

    p = plot(;
        xlabel    = L"\textrm{Approximation\ order\ }n",
        title     = L"\textrm{Cantor\ Filter\ Properties\ vs.\ Order\
}n",
        grid      = true,
        gridalpha = 0.3,
        size      = (850, 480),
        dpi       = 180,
    )

    bar!(p, collect(ns) .- 0.2, flat_measures;
        bar_width = 0.35,
        color      = :royalblue,
    )

```



```

        alpha      = 0.8,
        label      = L"\textrm{Flat\ region\ measure:\ }1-(2/3)^n",
        ylabel     = "Flat region measure",
        ylims      = (0, 1.1),
    )

    p2 = twinx(p)
    bar!(p2, collect(ns) .+ 0.2, peak_derivs;
        bar_width = 0.35,
        color      = :red3,
        alpha      = 0.8,
        label      = L"\textrm{Peak\ derivative:\ }(3/2)^n",
        ylabel     = "Peak derivative value",
    )

    savefig(p, joinpath(OUTDIR, "fig7_flat_region_growth.png"))
    println("✓ Figure 7 saved")
    return p
end

function main()
    println("=" ^ 60)
    println(" 칸토어 계단 함수 × Neural ODE 시뮬레이션 (Julia)")
    println("=" ^ 60)
    println()

    plot_fig1()
    plot_fig2()
    plot_fig3()
    plot_fig4()

    sol_no, sol_c3, sol_c5 = plot_fig5()
    plot_fig6(sol_no, sol_c3, sol_c5)

```

```
plot_fig7()

println()
println("=" ^ 60)
println("=" ^ 60)
for f in sort(readdir(OUTDIR))
    println("$f")
end
end

main()
```

평가요소	채점기준		점수
미분 계수 및 도함수의 시각화	상	적절한 코드를 선택하여 함수를 입력하였으며, 함수의 특징이 잘 드러나도록 시각화하였다.	8
	중	코드의 활용과 함수의 시각화 중 한 가지만 적절하게 활용하였다.	6.5
	하	코드의 활용과 함수의 시각화 두 가지 모두 적절하게 이루어지지 못했다.	3
부정적분 및 정적분의 시각화	상	탐구주제를 뒷받침하는 이론(함수/방정식)을 적절하게 선정하였고, 이론(함수/방정식)의 도함수를 유도하는 과정이 명확하다.	8
	중	탐구주제를 뒷받침하는 이론(함수/방정식) 선정과 이론(함수/방정식)의 도함수 유도 중 한 가지만 진행하였다.	6.5
	하	탐구주제를 뒷받침하는 이론(함수/방정식) 선정과 이론(함수/방정식)의 도함수 유도 두 가지 모두 적절하게 이루어지지 못했다.	3
동기와 느낀점 제시 여부	상	탐구 동기와 느낀점 모두 작성하였다.	4
	중	탐구 동기와 느낀점 중 하나만 작성하였다.	3
	하	탐구 동기와 느낀점 모두 작성하지 못하였다.	2